EY AI & Data Training

3-Hour Cohort Assessment

Duration: 180 minutes **Assessment type:** Hands-on + short concept questions

Section	Coverage	Time
A	Pandas / NumPy / Visualization / Data Types	60 min
B	Git + Cloud Fundamentals	30 min
C	PostgreSQL / SQL	90 min

Use the supplied data files and setup scripts. Keep your solutions simple and readable.

Section A - Data Handling and Data Types

Use: **urban_mobility.csv**, **equipment_events.json**, and **coastal_monitoring_log.txt**.

A1. Load urban_mobility.csv into a Pandas DataFrame and display the first five rows.

A2. Create a new column **Energy_Per_Km** = Energy_Used_kWh / Distance_Km.

A3. Display only Electric Van records where Maintenance_Flag is Yes.

A4. Group by Vehicle_Type and display average Distance_Km and total Trips_Count.

A5. Use NumPy to calculate the mean, maximum and minimum of Energy_Used_kWh.

A6. Create one bar chart showing average Energy_Used_kWh by Vehicle_Type using Matplotlib or Seaborn.

A7. Load equipment_events.json and print event_id, asset type, site and severity for every event.

A8. Display JSON records with temperature above 70. If actions is missing, display 'No actions recorded'.

A9. Read coastal_monitoring_log.txt and count total words and the number of times the word 'inspection' appears.

A10. Print only the lines containing either 'boardwalk', 'maintenance', or 'wind'.

Short concept: State which of the three supplied files is structured, semi-structured and unstructured, and give one reason for each.

Section B - Git and Cloud Fundamentals

Use the supplied files **distance_tool.py** and **weather_note.py**.

B1. Create a Git repository and add both Python files.

B2. Run git status, stage the files, create the first commit and push the repository.

B3. Create a branch named **feature-update**. Modify one Python file, commit the change and push the branch.

B4. Create a pull request from feature-update to main. You do not need to merge it if your platform permissions do not allow it.

Answer these briefly:

B5. What is the difference between cloud infrastructure and on-premises infrastructure?

B6. What is an Azure subscription?

B7. What is an Azure Resource Group?

B8. Arrange these in the correct hierarchy: Resource, Subscription, Resource Group.

B9. Why might a company place related Azure resources in the same Resource Group?

Section C - PostgreSQL / SQL

Create a database named **spare_parts_assessment**. Connect to it and run **spare_parts_setup.sql**.

Complete the following SQL tasks.

C1. Display all parts belonging to the Mechanical category.

C2. Display parts with unit_price above 1000, ordered from highest price to lowest.

C3. Find the average unit_price for each category.

C4. Find the total stock quantity for each category.

C5. Display every purchase order with supplier_name, order_date and status using a JOIN.

C6. Join purchase_orders, purchase_order_items and parts to display po_id, part_name, quantity and unit_price.

C7. Extend the previous query to calculate line_total = quantity × unit_price.

C8. Use a LEFT JOIN to display all suppliers, including suppliers that have never received a purchase order.

C9. Display only suppliers that have no purchase orders.

C10. Create a view named **purchase_order_summary** showing po_id, supplier_name, part_name, quantity, unit_price, line_total and status. Query the view for Pending orders.

C11. Create a function **calculate_part_discount(amount)**: if amount >= 2000 return 10% of amount, otherwise return 0. Use it with the parts table.

C12. Create a function **final_part_price(amount)** that returns amount minus calculate_part_discount(amount). Use it in a SELECT query.

C13. Create a stored procedure with parameters named **update_po_status** that accepts po_id and new_status and updates the matching purchase order. Call it once and verify the result.

C14. Create a stored procedure without parameters named **mark_old_pending_orders** that changes Pending orders dated before 2026-10-04 to Received. Run it and verify the results.

C15. Normalization question: Suppose supplier_name, supplier_city, part_name, part_category, po_id and quantity were all stored repeatedly in one large table. Identify at least two repeated facts and suggest how you would separate the data into tables.

Submission: Python scripts for Section A, Git evidence/screenshots for Section B, and one SQL file containing all Section C answers.